Task: Providing advice or solutions to a problem.

3 Different Prompts:

1.Concise: “ how to learn dsa”

Response: 1

✅ Step-by-Step Roadmap to Learn DSA

🔷 Step 1: Choose a Programming Language

- Best choices: **C++**, **Java**, or **Python**.
- Recommended for beginners in DSA: **C++** (more efficient for competitive coding, STL support).

🔷 Step 2: Learn Programming Basics

Before jumping into DSA, master the basics of programming:

- Variables and Data Types
- Input/Output
- Conditional Statements (if-else)
- Loops (for, while)
- Functions
- Arrays and Strings (basic operations)

📘 Resources:

- W3Schools / GeeksforGeeks: C++ Tutorial
- Practice: HackerRank, Coding Ninjas

🔷 Step 3: Start with Basic Data Structures

1. **Arrays**
2. **Strings**
3. **Linked Lists**
4. **Stacks**
5. **Queues**

💡 Learn:

- Implementation
- Time & Space Complexity
- Common Problems

📘 Practice: LeetCode Easy Problems

🔷 Step 4: Master Core Algorithms

1. Sorting (Bubble, Selection, Insertion, Merge, Quick)
2. Searching (Linear, Binary)
3. Recursion & Backtracking
4. Two Pointers & Sliding Window
5. Hashing (HashMaps, Sets)

📘 Practice: GeeksforGeeks DSA Sheet

🔷 Step 5: Advance to Complex Structures

1. Trees (Binary Tree, BST)
2. Heaps
3. Graphs (BFS, DFS, Dijkstra's)
4. Tries
5. Segment Trees & Fenwick Trees (for CP)

📘 Visualization Tools:

- Visualgo.net

🔷 Step 6: Solve DSA Problems Regularly

- LeetCode: for Product-Based Companies
- CodeStudio (Coding Ninjas)
- Codeforces, AtCoder, or CodeChef: for Competitive Programming

🧠 Use problem sheets:

- Striver's DSA Sheet
- Love Babbar 450 DSA Sheet

🔷 Step 7: Learn Time and Space Complexity

- Big O Notation
- Best/Worst/Average Cases
- Analyze each algorithm you write

🔷 Step 8: Mock Interviews & Contests

- Participate in contests on:
  - LeetCode (Weekly)
  - Codeforces / CodeChef
  - GFG Job-a-thon

🎯 Simulate interviews:

- Pramp
- InterviewBit

2.Medium: " how to learn dsa fast and easily using cpp "

Response: 2

🔷 Step 1: Master C++ Basics (2–3 days)

Understand:

- Variables, Data types
- Loops (for, while), Conditionals (if/else)
- Functions
- Arrays & Strings (basic usage)

📘 Resources:

- C++ W3Schools
- GeeksforGeeks C++

🔷 Step 2: Learn C++ STL (3–4 days)

Use STL to make coding fast and simple:

- `vector`, `pair`
- `map`, `set`, `unordered_map`
- `stack`, `queue`, `priority_queue`
- `sort()`, `binary_search()`

📘 Resource: [GFG STL Playlist]

🔷 Step 3: Start with Core DSA Topics (15–20 days)

Day Range	Topics	What to Focus
Day 1–3	**Arrays**	Reverse, Rotate, Subarrays, Kadane
Day 4–5	**Strings**	Palindrome, Anagram, Character Count
Day 6–8	**Linked List**	Reverse, Detect Loop, Merge Two LL
Day 9–10	**Stack & Queue**	Next Greater Element, Queue using Stack
Day 11–13	**Recursion + Backtracking**	Factorial, Subsets, N-Queens
Day 14–16	**Searching & Sorting**	Binary Search, Merge/Quick Sort
Day 17–19	**Hashing (Maps/Sets)**	Frequency count, Longest Subarray Sum
Day 20+	**2 Pointer & Sliding Window**	Target sum, Max subarray of size K

🛠 Practice on: LeetCode Easy

🔷 Step 4: Move to Advanced DSA (2–3 weeks)

Topic	Key Concepts
Trees	DFS, BFS, Height, Diameter, LCA
Binary Search Trees (BST)	Insertion, Deletion, Floor/Ceil
Heaps	Max/Min Heap, HeapSort, Priority Queue
Graphs	BFS, DFS, Dijkstra, Union-Find
DP (Dynamic Programming)	Fibonacci, Knapsack, LIS

📘 Best course: Striver's A2Z DSA Sheet

🔷 Step 5: Daily Practice (Compulsory)

- Practice 3–5 problems daily on:
  - **LeetCode**
  - **CodeStudio (Coding Ninjas)**
  - **GFG (Topic-wise problems)**

🚀 Extra Tips to Learn DSA Fast

1. **Don't memorize code** – understand logic.
2. **Use Pen & Paper** for dry runs and edge cases.
3. **Watch visual explanations** on YouTube for complex topics (e.g., Trees, Graphs).
4. **Solve by topics**, not randomly.
5. **Track progress** in Notion or Excel.

📅 Fast-Track 30-Day Learning Plan (DSA in 1 Month with C++)

Week	Focus
Week 1	C++ Basics + STL + Arrays, Strings
Week 2	Linked Lists, Stack, Queue, Recursion
Week 3	Sorting, Searching, Hashing, Sliding Window
Week 4	Trees, Graphs, DP + Mock Interviews

3. Detailed: "how to learn dsa completely, which language would be better ,that are all the topics to be know prerequisites how much time will be take to complete or learn dsa "

Response:3

Great! Let's break down everything you need to know to **learn DSA (Data Structures & Algorithms) completely** – including:

- 🔶 Best language choice
- 🔶 Complete topic list (basic to advanced)

- 🔸 Prerequisites
- 🔸 How much time it takes to complete
- 🔸 Best learning strategy

✅ 1. Which Language is Best for DSA?

Language	Pros	Cons	Best For
C++	Fast, STL library, widely used in competitive coding	Slightly complex syntax	DSA + Competitive Programming
Java	Object-oriented, large community support	Verbose code	Interviews, backend dev
Python	Simple syntax, fast to write	Slower, limited typing	Beginners, quick prototyping

✅ **Recommendation:**

- **C++** – best for learning DSA deeply and building logic
- **Python** – good if you're completely new and want to focus more on DSA than syntax
- **Java** – great if you're aiming for software development jobs

---

✅ 2. Prerequisites Before DSA

You must be comfortable with basic programming in your chosen language:

- Data Types & Variables
- Input / Output
- Conditional Statements (if-else)
- Loops (for, while)
- Functions
- Arrays & Strings (basic operations)

✅ 3. Complete DSA Topics (From Basics to Advanced)

🟢 Basic DSA Topics

Topic	Key Concepts
Arrays	Traversal, Reverse, Rotate, Kadane's Algorithm
Strings	Palindrome, Anagrams, Substrings
Linked List	Singly, Doubly, Reverse, Detect Cycle
Stack	Using array/LL, NGE, Balanced Brackets
Queue	Circular Queue, Deque, Sliding Window
Recursion	Factorial, Subsets, Permutations

🟡 Intermediate Topics

Topic	Key Concepts
Searching	Binary Search, Upper/Lower Bound
Sorting	Merge, Quick, Insertion, Counting
Hashing	Hash Maps, Sets, Frequency Count
Two Pointer	Pair Sum, Triplets, Duplicates
Sliding Window	Max subarray sum, K-sized window

🔵 Advanced DSA Topics

Topic	Key Concepts
Trees	Traversals, BST, LCA, Diameter
Graphs	BFS, DFS, Topo Sort, Dijkstra
Heap	Min/Max Heap, Priority Queue
Tries	Insert/Search, Prefix Matching
Greedy	Activity Selection, Huffman Coding
Backtracking	N-Queens, Sudoku Solver
Dynamic Programming	0/1 Knapsack, LIS, LCS
Segment Trees	Range Queries, Lazy Propagation
Disjoint Set	Union-Find, Path Compression

✅ 4. How Long Does It Take to Learn DSA Completely?

Learning Mode	Daily Time	Duration
Slow-paced (college students)	1–2 hrs/day	6–8 months
Moderate-paced (serious learners)	2–3 hrs/day	3–4 months
Fast-paced (intensive learning)	4–6 hrs/day	1–2 months

✅ **Consistent daily practice is more important than speed.**

✅ 5. Best Strategy to Learn DSA Completely

◆ Step-by-Step Plan

1. **Learn basic programming**
2. **Start DSA with Arrays & Strings**
3. **Follow a topic-wise roadmap** (use Striver's or Love Babbar's sheet)
4. **Solve 3–5 problems per day**
5. **Revise every week**
6. **Attempt mock tests / contests**
7. **Build notes + track mistakes**

◆ Resources to Follow

Platform	Use
GeeksforGeeks	Theory + Problems
LeetCode	Interview Practice
Striver's DSA Sheet	Topic-wise roadmap
Codeforces / AtCoder	Competitive coding
Visualgo.net	DSA visualizations

Brief understanding:

The changes made to the prompts significantly influenced the quality and relevance of the output. By refining the wording, adding specificity, and aligning the prompt more closely with the desired outcome, the AI was better able to generate accurate and contextually appropriate responses. For instance, prompts that included clear

instructions, examples, or a defined structure led to more focused and coherent answers. This worked particularly well when the task required a specific format, tone, or objective, such as summarizing a concept or generating code snippets.

What worked well was using direct, detailed prompts that minimized ambiguity. When the instructions clearly stated what to include or avoid, the model responded more precisely. Additionally, breaking down complex instructions into simpler parts helped in getting more refined outputs.

However, areas for improvement remain. Some prompts were still too broad or assumed prior knowledge, which led to vague or generic responses. To improve, prompts should be tested iteratively—tweaked based on feedback to reduce misunderstanding. Including the purpose of the task within the prompt can also guide the model toward a better output. Overall, thoughtful prompt design is crucial to leveraging AI tools effectively and should evolve through experimentation and context-aware revisions.

PYTHON ASSIGNMENT:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Cannot divide by zero."
    return a / b

def power(a, b):
    return a ** b

def get_number(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Invalid input. Please enter a valid number.")

def calculator():
    while True:
        print("\n===== Basic Calculator =====")
        print("Choose an operation:")
        print("1. Addition (+)")
        print("2. Subtraction (-)")
        print("3. Multiplication (*)")
        print("4. Division (/)")
        print("5. Power (^)")
        print("6. Exit")
```

```
    choice = input("Enter your choice (1-6): ")

    if choice == '6':
        print("Exiting calculator. Goodbye!")
        break

    if choice not in ['1', '2', '3', '4', '5']:
        print("Invalid choice. Please select a valid option.")
        continue

    num1 = get_number("Enter the first number: ")
    num2 = get_number("Enter the second number: ")

    if choice == '1':
        print("Result:", add(num1, num2))
    elif choice == '2':
        print("Result:", subtract(num1, num2))
    elif choice == '3':
        print("Result:", multiply(num1, num2))
    elif choice == '4':
        result = divide(num1, num2)
        print("Result:", result)
    elif choice == '5':
        print("Result:", power(num1, num2))

# Run the calculator
calculator()
```

Output

```
Enter your choice (1-6): 6
Exiting calculator. Goodbye!
```

<u>ADDITIONAL TASK</u>

PPT ON ADVANCED PROMPT ENGINEERING TECHNIQUES

https://docs.google.com/presentation/d/1HoLSpLKQY6goVAE1bMaDSdm-GFHIi4gp/edit?usp=drivesdk&ouid=114051623539409857092&rtpof=true&sd=true

Prompt Design Experiment

1. Objective

The aim of this experiment is to compare three advanced prompt engineering techniques — Zero-shot, Few-shot, and Chain-of-Thought — using a complex task. We will analyze the performance of each method using a math-based problem and document the outputs and observations.

2. Selected Task: Math Word Problem

Problem Statement:
A train travels 180 km in 3 hours. How long will it take to travel 300 km at the same speed?

This task involves applying the formula:
Speed = Distance / Time
Time = Distance / Speed

3. Prompt Designs
3.1 Zero-Shot Prompt

"A train travels 180 km in 3 hours. How long will it take to travel 300 km at the same speed?"

3.2 Few-Shot Prompt
"Here are some examples of solving similar problems:

Example 1:
A car travels 120 km in 2 hours. How long will it take to travel 240 km at the same speed?
→ Speed = 120/2 = 60 km/hr
→ Time = 240/60 = 4 hours

Example 2:
A bike covers 90 km in 1.5 hours. How long to cover 180 km at the same speed?
→ Speed = 90/1.5 = 60 km/hr
→ Time = 180/60 = 3 hours

Now solve this:
A train travels 180 km in 3 hours. How long will it take to travel 300 km at the same speed?"

3.3 Chain-of-Thought Prompt
"Let's solve this step-by-step:
A train travels 180 km in 3 hours.
Step 1: Find the speed of the train.
Step 2: Use that speed to find how long it takes to go 300 km.
Step 3: Give the final answer."

4. AI Model Used

Model: GPT-4
Platform: ChatGPT
Date of Experiment: [Insert today's date]

5. Responses and Observations

5.1 Zero-Shot Response

Answer: The train will take 5 hours to travel 300 km at the same speed.

Observation:

✅ Correct
❌ No explanation or reasoning

⚠ Not suitable for educational or complex reasoning tasks

5.2 Few-Shot Response

> Examples Recapped
Speed = 180 / 3 = 60 km/h
Time = 300 / 60 = 5 hours

Observation:

✅ Correct
✅ Provides structure
✅ Easier to understand due to examples
⚠ Requires pre-curated examples

5.3 Chain-of-Thought Response

> Step-by-step:
Step 1: Speed = 180 / 3 = 60 km/h
Step 2: Time = 300 / 60 = 5 hours
Step 3: Final answer = 5 hours

Observation:

✅ Correct
✅ Transparent reasoning
✅ Ideal for logic/math problems
⚠ Slightly longer prompt/output

6. Comparison Table

Prompt Technique	Accuracy	Reasoning Clarity	Best Use Cases	Limitations
Zero-Shot	✅	❌	Quick factual questions	Lacks step explanation
Few-Shot	✅	✅	Pattern-based tasks, NLP	Needs carefully picked examples
Chain-of-Thought	✅	✅✅	Reasoning-heavy tasks, logic/math	Slightly verbose, longer prompts

7. Conclusion

All three techniques correctly solved the problem, but their effectiveness varies by context:
Zero-shot is fast but lacks transparency.
Few-shot provides structure and improves accuracy for patterned tasks.
Chain-of-Thought is most effective for reasoning and educational explanations.

Understanding when and how to apply each technique can significantly improve AI performance and user satisfaction.